Wallet SDK documentation

- Cantor8 WalletConnect SDK
  - When to use C8 WalletConnect SDK
  - Standard transfers
  - Custom transaction types
- Preview how it works
- Sequence flow diagram
- Technical Documentation
  - Overview
  - Installation
  - Quick start
  - Core concepts
  - API reference
  - Events
  - Error handling
  - Networks available

Cantor8 WalletConnect SDK

Lightweight SDK for dApp developers to integrate the C8 Wallet via a popup window and `postMessage` communication. Built for the Canton network by Cantor8.

When to use C8 WalletConnect SDK

C8 WalletConnect SDK is built for DeFi applications running on the Canton network that need a ready-made wallet layer — without building auth, key management, or transaction signing from scratch.

Standard transfers

Out of the box, the SDK supports connecting a Canton wallet, resolving instruments and accounts, and submitting transfers between parties. This covers the majority of DeFi use cases: payments, settlements, and asset movement across Canton participants.

Custom transaction types

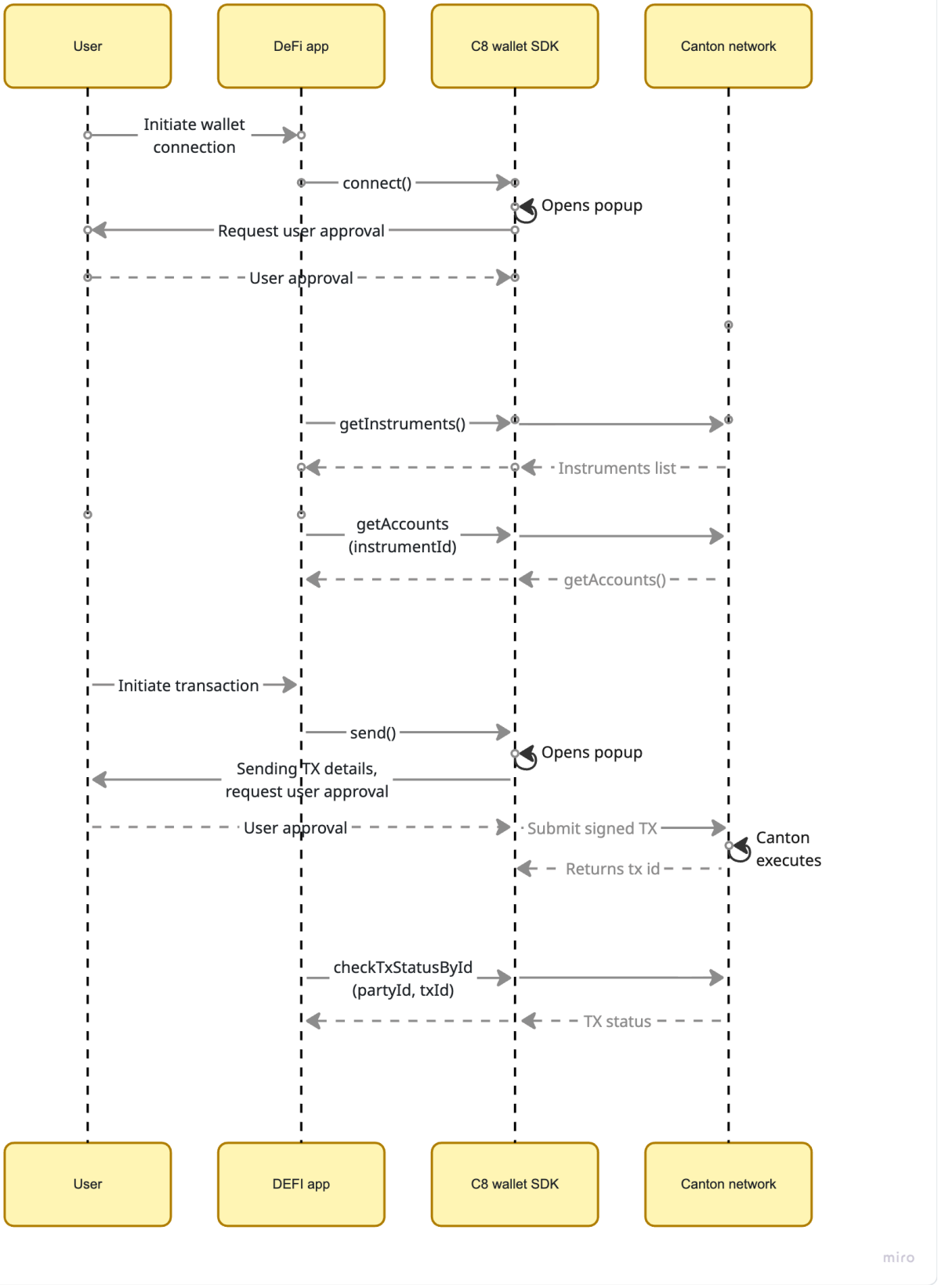
Canton's smart contract model (Daml) allows for arbitrarily complex workflows — multi-party agreements, conditional transfers, bespoke financial instruments. If your application requires transaction types beyond standard transfers, the SDK can be extended to support them. This involves aligning on your contract and workflow requirements with the Cantor8 team, who will implement the corresponding support on the C8 backend.

To discuss custom integration, contact us.

Preview how it works

//tbd (video)

Sequence flow diagram


User
DeFi app
C8 wallet SDK
Canton network
Initiate wallet connection
connect()
Opens popup
Request user approval
User approval
getInstruments()
Instruments list
getAccounts (instrumentId)
getAccounts()
Initiate transaction
send()
Opens popup
Sending TX details, request user approval
User approval
Submit signed TX
Canton executes
Returns tx id
checkTxStatusById (partyId, txId)
TX status
User
DEFI app
C8 wallet SDK
Canton network
miro

Technical Documentation

Overview

`C8WalletProvider` handles the full wallet lifecycle — connection, instrument and account resolution, transfer submission, and status polling — without requiring dApp developers to manage Canton-specific cryptography or ledger communication directly. All wallet interaction happens through a secure popup window using the browser's native `postMessage` API.

Installation

`npm install cantor8-wallet-connect-sdk.tgz`

Quick start

1. Initialize the provider

js

```
const c8 = new C8WalletProvider({
  dappUrl:  window.location.href,
  dappName: "My DeFi App",
  network:  "devnet"   // "devnet" | "mainnet"
});
```

2. Subscribe to events

js

```
c8.on('connected',       (e) => console.log('Connected', e));
c8.on('disconnected',    (e) => console.log('Disconnected',
e));
c8.on('accountChanged',  (e) => console.log('Balance change
d', e));
c8.on('txInitiated',     (e) => console.log('TX initiated',
e));
```

```
c8.on('txChanged',       (e) => console.log('TX changed',
e));
```

3. Connect and run a transfer

js

```
await c8.connect();

// Fetch available instruments (Canton tokens / assets)
const { instruments } = await c8.getInstruments();
const instrumentId = instruments[0]?.instrumentId;

// Fetch accounts for that instrument
const { accounts } = await c8.getAccounts(instrumentId);
const senderPartyId          = accounts[0]?.partyId;
const senderHoldingInstrId   = accounts[0]?.holdings[0]?.inst
rumentId ?? instrumentId;

// Submit a transfer
const { txId } = await c8.send({
  senderPartyId,
  instrumentId: senderHoldingInstrId,
  amount:           1.23,
  receiverPartyId: 'Party::Receiver',
  // memo and metadata are optional — see API reference
});

console.log('Transaction submitted:', txId);
```

Core concepts

Canton network

C8 Wallet is built on Canton, a privacy-enabled blockchain designed for institutional finance. Rather than Ethereum addresses, Canton uses `partyId` strings

to identify participants, and `instrumentId` to identify tokenized assets. Your dApp works with these identifiers throughout the SDK.

Popup + postMessage architecture

The SDK facilitates interaction between the dApp and the wallet using two primary channels, depending on the nature of the request:

- `window.postMessage` **(Interactive Actions):** All actions requiring explicit user consent, auth, or cryptographic signatures — such as connecting an account or signing transactions — are handled via `window.postMessage`.
- **HTTP API (Read-Only Data):** Standard data retrieval and "reading" actions that do not require user intervention are performed through direct HTTP API calls.

Instruments and holdings

An instrument is a Canton asset type (e.g. a tokenized currency or security). A holding is a specific account's balance of that instrument. The typical flow is: `getInstruments()` → pick an `instrumentId` → `getAccounts(instrumentId)` → pick a `partyId` and holding.

API reference

`new C8WalletProvider(config)`

Parameter	Type	Required	Description
`dappUrl`	`string`	-	
`dappName`	`string`	✓	DappName, must be equal to dApp
`network`	`string`	-	Canton network: `"devnet"`, or `"mainnet"`

`connect(): Promise<void>`

Opens the C8 Wallet popup and establishes the `postMessage` channel. The user sees a connection approval screen in the wallet. Resolves when the user approves.

`disconnect(): Promise<void>`

Closes the wallet popup and tears down the `postMessage` channel. Emits the `disconnected` event.

`status(): Promise<{ connected: boolean, walletVersion?: string }>`

Returns the current connection state and the wallet's version string if connected.

`getInstruments(): Promise<InstrumentListPayload>`

Returns the list of Canton instruments available in the connected wallet.

ts

```
type InstrumentListPayload = {
  instruments: Array<{
    instrumentId: string
    name: string
    symbol: string
  }>
}
```

Returns accounts and their holdings. If `instrumentId` is provided, filters to accounts holding that instrument.

`getAccounts(instrumentId?: string): Promise<AccountListPayload>`

ts

```
type AccountListPayload = {
  accounts: Array<{
    partyId: string
    holdings: Array<{
      instrumentId: string
      balance: number
      balanceUsd: number
    }>
  }>
}
```

`send(input): Promise<SubmitTransferResultPayload>`

Submits a transfer on the Canton ledger. Opens an approval screen in the wallet popup. Resolves with `{ txId }` once the user approves and the transaction is initiated.

Field	Type	Required	Description
`senderPartyId`	`string`	✓	Canton partyId of the sender
`instrumentId`	`string`	✓	Asset to transfer
`amount`	`number`	✓	Transfer amount
`receiverPartyId`	`string`	✓	Canton partyId of the recipient
`memo`	`string`	—	Optional text memo attached to the transfer
`metadata`	`Record<string, string>`	—	Arbitrary key-value metadata

`checkTxStatusById(input): Promise<TransferStatusResponsePayload>`

Polls the Canton ledger for the current status of a previously submitted transaction.

Field	Type	Required	Description
`txId`	`string`	✓	Transaction ID returned by `send()`

Events

All events are subscribed to with `c8.on(event, handler)`.

Event	Payload	When it fires
`connected`	—	User approves the connection in the popup
`disconnected`	—	`disconnect()` called or popup closed
`accountChanged`	updated account data	Balance or holdings change detected
`txInitiated`	`{ txId }`	Transfer submitted to Canton
`txChanged`	status update	Transaction status changes on the ledger

Error handling

js

```
try {
  const { txId } = await c8.send({ ... });
} catch (err) {
  switch (err.code) {
```

```
   case 'USER_REJECTED':      // user dismissed the approval
popup
    case 'INSUFFICIENT_FUNDS': // holding balance too low
    case 'POPUP_BLOCKED':      // browser blocked the popup w
indow
    case 'NOT_CONNECTED':      // connect() was not called fi
rst
    case 'FAILED_TO_FETCH_TRANSFER_STATUS':
      case 'INITIALIZATION_FAILED':
    case 'EXECUTION_FAILED':
    case 'TRANSFER_PREPARATION_FAILED':
    case 'FETCH_INSTRUMENTS_FAILED':
    case 'FETCH_ACCOUNTS_FAILED':
    case 'CHECK_TX_STATUS_FAILED':
  }
}
```

Networks available

Network	Usage
`devnet`	Local development and integration testing
`mainnet`	Production Canton network